

Data Binding in Angular

Data binding is the core concept of Angular and used to define the communication between a component and the view. It is a technique to link your data to your view. In simple words, you can say that data binding is a communication between your typescript code of your component and your template(View) which user sees. It makes easy to define interactive applications without worrying about pushing and pulling data.

Data binding can be either one-way data binding or two-way data binding.

One-way databinding

One way databinding is a simple one way communication where HTML template is changed when we make changes in TypeScript code(code File).

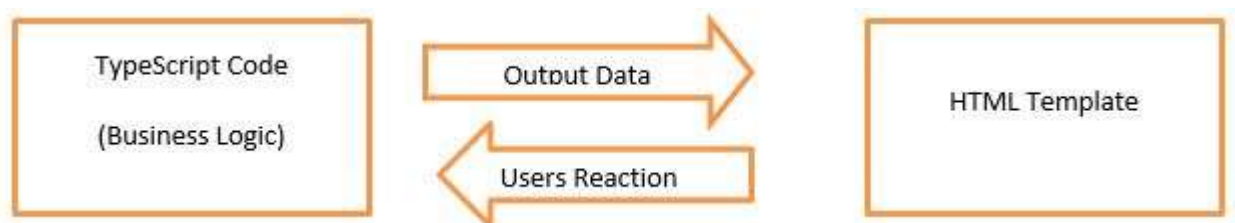
Or

In one-way databinding, the value of the Model is used in the View (HTML page) but you can't update Model from the View. Angular Interpolation / String Interpolation, Property Binding, and Event Binding are the example of one-way databinding.

Two-way databinding

In two-way databinding, automatic synchronization of data happens between the Model and the View. Here, change is reflected in both ends. Whenever you make changes in the Component(Model), it will be reflected in the View and when you make changes in View, it will be reflected in Component(Model).

This happens immediately and automatically, ensures that the HTML template and the TypeScript code are updated at all times.



Angular provides four types of data binding and they are different on the way of data flowing.

- String Interpolation

- [Property Binding](#)
 - [Event Binding](#)
 - [Two-way binding](#)
-

String interpolation

String Interpolation is a **one-way databinding technique** which is used to output the data from a TypeScript code to HTML template (view). It uses the template expression in **double curly braces** to display the data from the component to the view.

For example:

```
{{ data }}
```

String interpolation adds the value of a property from the component:

Syntax:

1. `<li>Name: {{ name }}</li>`
 2. `<li>Email: {{ email }}</li>`
-

Property Binding

Property Binding is also a **one-way data binding** technique. In property binding, we bind a property of a DOM element to a field which is a defined property in our component TypeScript code.

For example:

```
<img [src]="imgUrl"/>
```

Syntax:

1. `<input type="email" [value]="user.email">`

Event Binding

In Angular , event binding is used to handle the events raised from the DOM like button click, mouse move etc. When the DOM event happens (eg. click, change, keyup), it calls the specified method in the component. In the following example, the cookBacon() method from the component is called when the button is clicked:

For example:

1. `<button (click)="cookBacon()"></button>`

\$event Payload

DOM Events carries the event payload. I.e the information about the event. We can access the event payload by using \$event as an argument to the handler function.

```
1
2 <input (input)="handleInput($event)">
3 <p>You have entered {{value}}</p>
4
```

And in the component

```
1
2 value=""
3 handleInput(event) {
4   this.value=event.target.value;
5 }
6
```

Note: The properties of a \$event object vary depending on the type of DOM event. For example, a mouse event includes different information than an input box editing event.

Remember you need to use the variable as \$event in the Template statement. Example handleInput(\$event). Otherwise, it will result in an error

Two-way Data Binding

We have seen that in one-way data binding any change in the template (view) were not be reflected in the component TypeScript code. To resolve this problem, Angular provides two-way data binding. The two-

way binding has a feature to update data from component to view and vice-versa.

In two way data binding, property binding and event binding are combined together.

Syntax:

1. [(ngModel)] = "[property of your component]"

Note: For two way data binding, we have to enable the ngModel directive. It depends upon FormsModule in angular/forms package, so we have to add FormsModule in imports[] array in the AppModule.